

Ninditya Salma Nur Aini

Data Analyst · Business Intelligence · Customer Care Analytics

Yogyakarta, Indonesia · +62 895 3712 90495 · ninditya.sna025@gmail.com
linkedin.com/in/ninditya-salma-nur-aini · ninditya.github.io

Hiring Team, Data & Business Intelligence

12 August 2026

Traveloka
BSD City, Indonesia

Subject: Application for Data Analyst, Customer Care (Job ID MJ000205)

Dear Hiring Team,

I am applying for the Data Analyst position supporting Customer Care product and operations. Most data analysts learn customer care metrics after they join. I have spent four years inside those operations. At PT Infomedia Nusantara (Telkom Group) I analyzed and reported on chatbot, omnichannel, and CRM data covering roughly 1,000 customer interactions per day, and at Flip I owned the quality metrics for an LLM/RAG-based customer support system, contributing to **65% CSAT**, a **3% fallback rate**, and unidentified utterances reduced to below **12%**. I already understand what deflection, containment, and fallback cost an operations team, which is the context this role starts from rather than builds toward.

My day-to-day work maps directly to the responsibilities listed. I run deep-dive analyses in SQL and Python on customer interaction data to find behavioral patterns, service gaps, and operational bottlenecks, and I build and maintain the dashboards that track them, using Looker Studio, Metabase, and Grafana. I have delivered these findings to Product, Engineering, and Process Improvement stakeholders as release and process decisions, not as reports. On the LLM side, the role asks for analysts who can evaluate where AI actually improves customer care efficiency. My Banking Intent Router project is exactly that question answered with numbers: a four-tier confidence-based routing system built on SBERT and LinearSVC that resolves high-confidence queries with a lightweight classifier and escalates only ambiguous cases to an LLM, holding **0.93 Macro F1** while reducing LLM inference cost by **88%** at roughly 50ms latency. My published sentiment analysis benchmark reaches the same conclusion from the other direction, where conventional ML outperformed a zero-shot GenAI approach on accuracy, latency, and cost simultaneously.

I want to be straightforward about fit. My cloud experience covers Vertex AI, Google Cloud Storage, Microsoft Fabric, and containerized deployment with Docker and FastAPI, and I have built and tested an end-to-end ETL pipeline with PostgreSQL and the Google Sheets API, but I have not yet owned an orchestrated warehouse pipeline in production. That is the one area where I would be ramping up rather than arriving fluent, and given my SQL and Python foundation I expect that ramp to be short. Everything else the role asks for, particularly the customer care domain knowledge listed as an advantage, is where I am strongest. I am available for the contractor engagement and would welcome the chance to discuss how I can contribute to the Customer Care analytics roadmap.

Thank you for your time and consideration.

Sincerely,

Ninditya Salma Nur Aini

ninditya.github.io · github.com/ninditya

Project Portfolio

Prepared for: Data Analyst, Customer Care · Traveloka · Job ID MJ000205

This document selects **eight pieces of work** from a wider portfolio, ordered by relevance to the Customer Care Analytics scope of this role rather than by recency. Each case study states the business problem, the analytical approach, and the measurable outcome. Every project is publicly accessible; live case study or repository links are included. Work performed under employment at Flip is described at the methodology level only, since the production system and its data are confidential.

01	LLM Chatbot Evaluation Framework	Flip · Production · 65% CSAT · 3% fallback
02	Banking Intent Router	0.9305 F1 · 88% cost cut · FastAPI + Grafana
03	ETL Fashion Pipeline	3 destinations · pytest · MagicMock
04	Talent Match Intelligence	17-CTE SQL · PERCENTILE_CONT · Streamlit
05	Student Dropout Early Warning System	92.42% acc · 97.27% ROC-AUC · Looker Studio
06	Employee Attrition Early Warning System	86.11% recall · Metabase · Docker
07	CineTrack Cinema Analytics Backend	PostgreSQL CTEs · Fastify · Redis
08	Credit Risk Modeling — LendingClub	466K loans · SHAP · cost-based threshold

Ninditya Salma Nur Aini

Yogyakarta · Data Analyst · Customer Care Analytics · SQL & BI Reporting

ninditya.sna025@gmail.com · +62 895-3712-90495
[ninditya.github.io](#) · [github.com/ninditya](#)

Role Requirement Mapping

Evidence index by case number

Each row below maps a stated requirement of the role to the specific case study in this document that demonstrates it. Case numbers refer to the eight case studies that follow.

Customer care operations & KPI understanding	Case 1. Four years of chatbot, omnichannel, and CRM operations at Infomedia and Flip.
Actionable insight delivered independently	Cases 1, 5, 6, 8. Driver analysis translated into intervention, retention, and underwriting recommendations.
SQL, spreadsheet, Python (2+ years)	Cases 3, 4, 7, 8. 17-CTE analytical SQL, ETL transformation logic, cinema revenue analytics, 466K-row Python modeling pipeline.
Dashboarding with BI tools (2+ years)	Cases 5, 6. Looker Studio and Metabase dashboards, plus Grafana in production operations.
Data pipelines in cloud environments	Case 3. End-to-end ETL to PostgreSQL and Google Sheets API, pytest-verified. Vertex AI and Microsoft Fabric via certification.
LLM knowledge and AI/ML deployment	Cases 1, 2. LLM evaluation methodology and cost-based routing architecture.
Advanced techniques, segmentation, key drivers	Cases 4, 5, 6, 8. Percentile benchmarking, SMOTE, SHAP driver identification, grade × tenor risk segmentation.

LLM Chatbot *Evaluation Framework*

≡ Case study – ninditya.github.io/llm-eval-qa ↗ Banking Router – evaluation/ ↗ HealthTruth-AI – evaluation/			
65% CSAT contributed via release gating	<12% Unidentified utterances reduced from higher baseline	3% Fallback rate sustained through dataset tuning	5 Metrics tracked per release

Context: Flip operates an LLM and RAG based customer support chatbot handling payment, transfer status, receipt verification, and refund dispute flows. Release quality had to be measurable before deployment, not diagnosed after complaints. **Problem:** Benchmark accuracy did not predict real user experience. The team needed operational metrics that connected model behavior to customer care outcomes.

EVALUATION APPROACH

STRUCTURED SAMPLING → BEHAVIORAL ANALYSIS → RELEASE GATE			
MEASURE	Structured sampling & evaluation workflow Hallucination rate, answer grounding, retrieval effectiveness, intent accuracy, and fallback triggers measured across releases — not a single aggregate accuracy score.		5 metrics
ANALYZE	Interaction dataset analysis in Python & SQL Surfaced behavioral patterns, labeling inconsistencies, and unhandled intents from real customer conversations, feeding directly back into dataset optimization.		Python + SQL
GATE	Release-over-release comparison dashboards Monitoring dashboards in Grafana and Metabase with acceptance criteria defined jointly with Product, Engineering, and Process Improvement — degradation caught before deployment.		Grafana · Metabase

METHODOLOGY DEMONSTRATED ON PUBLIC RAG SYSTEMS

HealthTruth-AI

Grounded answers, not model opinions.

A fact-checker for herbal medicine claims with personalized health context.

FastAPI · LangGraph · Odrant · Gemini 2.5 Flash · Streamlit

2 MODES

- Tanya (Ask)
- Cek Fakta (Fact-Check)

3 DEPTHS (Ask Mode)

- Ringkas 2-3 sentences
- Detail Full explanation
- Sumber With citations

The real question: Is this claim true for this person, given their specific health context?

THE CORE PAIN POINT

How do we make a RAG system stay useful when the LLM can't answer — and personalize responses without leaking user health data?

- LLM quota exhaustion is inevitable (429 / 503 / RESOURCE_EXHAUSTED)
- Retrieval sometimes returns nothing relevant
- Health data is sensitive and must not be stored on the server
- Generic answers are dangerous. Advice must consider the user's conditions, allergies, and medications.

If the system fails here, users get nothing — or worse, they get hallucinations.

SYSTEM ARCHITECTURE (RETRIEVAL-FIRST, RELIABILITY BY DESIGN)

- User (Streamlit)
- Profile Context Injection (Phyllis pregnant? Demografi: Pria, 58 thn. Riwayat: Diabetes Tipe 2, Hipertensi. Alergi: Jambu. Obat rutin: Warfarin.)
- Embedding (Gemini) 3272-dim SEMANTIC_SIMILARITY [0.21, -0.17, 0.32, ...]
- Vector Search (Odrant Cloud) Cosine Similarity score_threshold = 0.4 top_k = 3 Deduplication on payload text
- LangGraph Workflow (3 Nodes) Conditional Routing
- Answer / Fallback (Gemini 2.5 Flash) Mode-specific prompt + XML isolation + Sanitized input Fact-Check Mode: JSON schema enforced
- Response JSON { answer, context_used, intent_rec, fact_check: { status, summary, explanation_sources }}

PERSONALIZATION WITHOUT STORING DATA: All personalization happens on the client (Streamlit). The API is stateless. Example Profile Context: [Phyllis pregnant? Demografi: Pria, 58 thn. Riwayat: Diabetes Tipe 2, Hipertensi. Alergi: Jambu. Obat rutin: Warfarin. Peringatan: Apakah asupan buah bisa mengganggu status untuk kolestrol?] Retrievial Impact: Retrieval logs respects the user + question → more relevant matches. Generation Impact: LLM asks the context + answers are personalized and safer.

DESIGN DECISION: NEVER CRASH ON QUOTA EXHAUSTION

Layered Fallback Chain

- Happy Path: Gemini 2.5 Flash answers with full prompt
- Fallback (Streamlit): Raw retrieval returned as answer (lm_fallback: true)
- Error (Odrant): HTTP 429 + retry message to user

Why this matters

- Free-tier LLM APIs have daily limits. Users still get something useful, not a 500 error.
- Client can show a warning banner (lm_fallback: true).

KEY IMPACT

- Reliability First** The system stays useful even when the LLM can't help.
- Grounded & Safe** Answers must come from trusted sources, not model opinions.
- Personalized but Private** Health context improves accuracy without storing sensitive data.
- Production Ready** Persistent vector store, modular architecture, graceful degradation.

TRUSTED KNOWLEDGE BASE BPOM, IDI, WHO, Kemendkes

DEPLOYMENT Railway 2 Services · Docker

LIVE DEMO [health-truth-chatbot.up.railway.app](#)

SOURCE CODE [View on GitHub](#)

HealthTruth-AI — one of two publicly built RAG systems (alongside the Banking Intent Router, Case 2) used to demonstrate this identical evaluation methodology, since Flip's production system and data are confidential. Retrieval-first architecture with a layered fallback chain designed never to fail silently — the same principle behind fallback-rate monitoring in production.

RESULT Contributed to 65% CSAT , unidentified utterances reduced to below 12% , and a 3% fallback rate sustained through continuous dataset optimization. Degradation was caught pre-release rather than through customer complaints.	CROSS-FUNCTIONAL LOOP Acceptance criteria for each release were defined jointly with Product, Engineering, and Process Improvement — evaluation output was consumed as a release decision, not a report.	CONFIDENTIALITY NOTE Flip production data and system internals are confidential. The linked write-up applies the identical sampling, grounding, and fallback-monitoring methodology to two RAG systems owned outright.
--	--	--

Stack: Python · SQL · structured sampling · Google Workspace · Grafana · Metabase

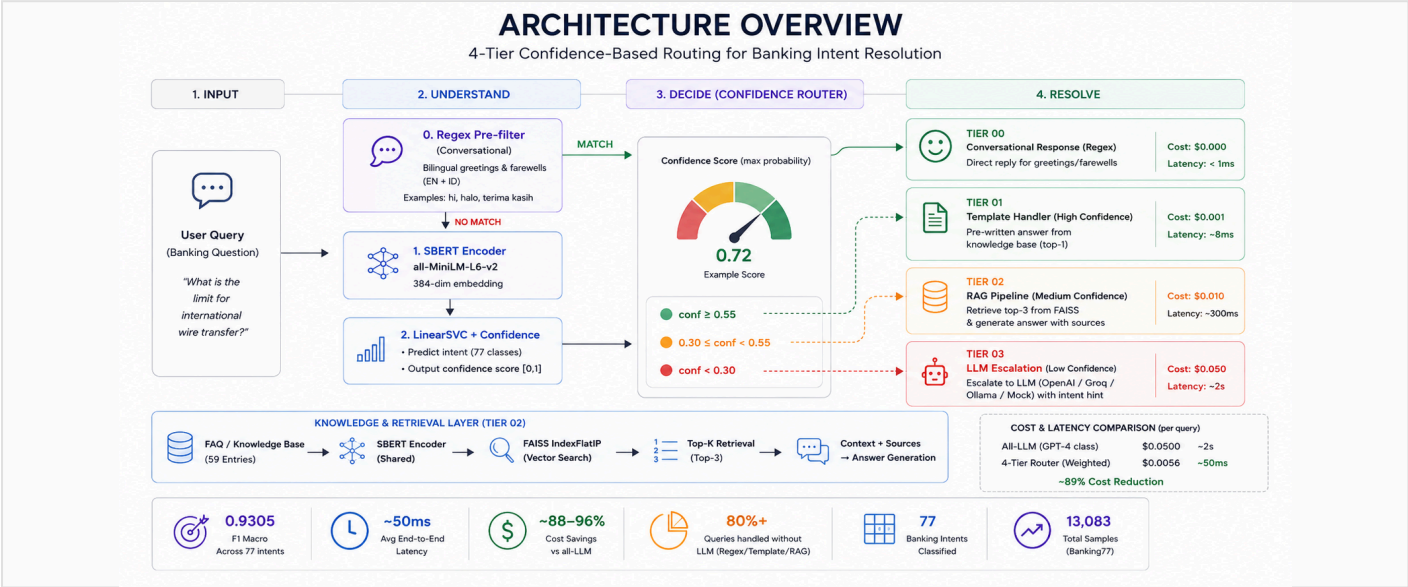
Banking Intent Router

➤ Live Demo — Hugging Face Spaces	🔗 GitHub Repository	≡ Case study
-----------------------------------	---------------------	--------------

0.9305 Macro F1 across 77 banking intents	88% LLM inference cost cut vs all-LLM baseline	~50ms Avg end-to-end routing latency	4 Confidence tiers tunable threshold
--	---	---	---

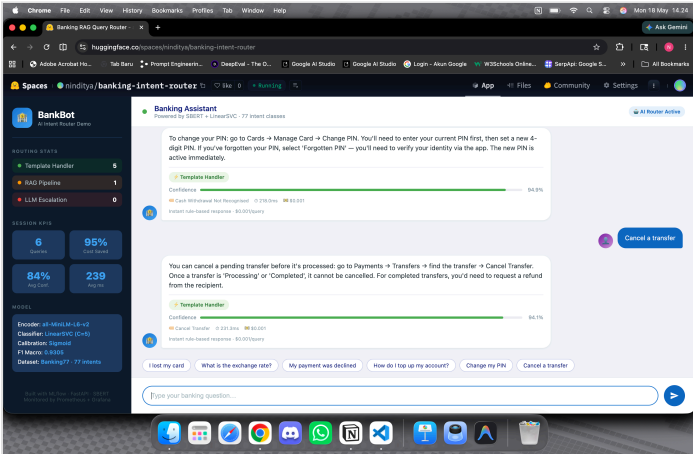
Context: Routing every customer query to an LLM is the default architecture and the most expensive one. In a customer care setting, a large share of incoming queries are repetitive and do not require generative reasoning. **Problem:** How much LLM spend can be removed without degrading answer quality, and where exactly should the escalation threshold sit?

ARCHITECTURE — 4-TIER CONFIDENCE-BASED ROUTING

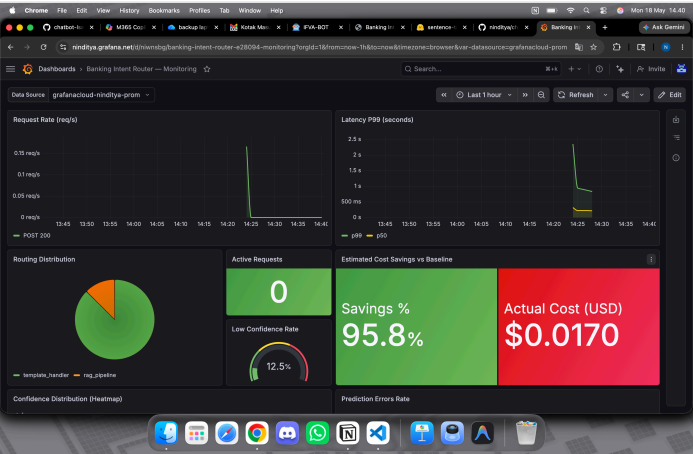


Pipeline: queries are classified by an SBERT embedding + LinearSVC classifier, then resolved by regex filter, template response, RAG retrieval, or full LLM call depending on confidence score — each tier priced and latency-profiled independently.

LIVE DEMO & PRODUCTION MONITORING



Live chat demo: each response is tagged with its routing tier, confidence score, latency, and per-query cost — the cost/quality trade-off made visible to the end user.



Grafana Cloud dashboard: request rate, P99 latency, routing distribution, and estimated cost savings vs. an all-LLM baseline — the same monitoring pattern used for release gating in Case 1.

RESULT 0.9305 Macro F1 maintained while LLM inference cost fell 88% , at approximately 50ms routing latency. The threshold is tunable, making the accuracy vs. cost trade-off an explicit business decision rather than an implicit one.	WHY THIS MATTERS FOR CUSTOMER CARE This is the exact question a Customer Care analytics function needs answered: where does AI actually reduce cost-to-serve without degrading CSAT, and where should human/LLM/rule-based handling boundaries sit.	SERVED IN PRODUCTION SHAPE FastAPI service with Prometheus/Grafana monitoring on confidence distribution and routing drift — deployable observability, not a notebook result.
--	---	---

Stack: Python · SBERT · LinearSVC · FastAPI · Grafana · Docker

ETL Fashion Pipeline

↗ GitHub Repository	≡ Case study
---------------------	--------------

50 Pages scraped per pipeline run	3 Load destinations CSV · PostgreSQL · Sheets	pytest Every stage MagicMock-verified	2 Validation rules schema + currency conversion
--	--	--	--

Context: Analytics-ready data rarely arrives clean. This project builds the extraction and transformation layer end to end, with the reliability checks that keep a pipeline trustworthy in production. **Problem:** Deliver validated, multi-destination data from an unstructured web source without silent failures in transformation logic.

PIPELINE ARCHITECTURE — EXTRACT → TRANSFORM → LOAD

MODULAR, INDEPENDENTLY TESTABLE LAYERS			
EXTRACT	BeautifulSoup scraper — 50 pages Scrapes fashion product cards page by page. A broken card is caught and skipped rather than crashing the run, so a single malformed page never loses the rest of the batch.		50 pages
TRANSFORM	USD → IDR conversion & schema validation Type conversion, currency normalization, and schema checks applied before load — invalid rows are rejected explicitly rather than silently coerced.		Schema-checked
LOAD	3-destination writer — CSV, PostgreSQL, Sheets Each destination write is independently guarded — a Google Sheets API failure does not block the CSV or PostgreSQL writes from completing.		3 destinations

TESTING STRATEGY — MAGICMOCK, NO LIVE DEPENDENCIES

The test suite uses MagicMock throughout — no real HTTP requests, no real database, no real Google Sheets API. Every stage of the pipeline is covered by pytest, so transformation and validation logic is verified without needing a live database or server running.

TEST	WHAT IT COVERS
test_extract	Field parsing · broken HTML handling
test_transform	Currency conversion · schema validation
test_load	Per-destination write & error isolation

RESULT Reproducible pipeline with full test coverage on transformation and validation, producing analytics-ready datasets across three consumption surfaces (CSV, PostgreSQL, Google Sheets).	RELEVANCE TO THIS ROLE Demonstrates the pipeline discipline requested for cloud data-pipeline work: validated data delivery to PostgreSQL and a Sheets-based reporting surface, with tests that catch failure modes before they reach a dashboard.	DESIGN PRINCIPLE Currency exchange rate is read from an environment variable with a safe default — not hardcoded — allowing rate updates without a code change.
--	---	--

Stack: Python · PostgreSQL · Google Sheets API · BeautifulSoup · pytest · MagicMock

2,010 Candidate records benchmarked	17 SQL CTEs in matching query	PERCENTILE_CONT Benchmark threshold function	100% Score components traceable to an SQL rule
--	----------------------------------	---	---

Context: Hiring decisions were made against job descriptions rather than against the measurable profile of people who already perform well in the role.

Problem: Benchmark 2,010 candidate records against top-performer characteristics using SQL alone, with logic transparent enough to be audited by non-technical stakeholders.

SQL MATCHING ALGORITHM ARCHITECTURE — 17 CTEs

PostgreSQL · 17-CTE Query Chain		
PHASE 1	Top-performer benchmark extraction Filter top-rated employees, compute PERCENTILE_CONT for continuous scores and MODE() for categorical traits. Output: a benchmark profile object.	CTEs 1–4
PHASE 2	Gap scoring per candidate For each of 2,010 candidates: compute distance from benchmark on each pillar, normalize to a 0–1 scale, apply inverse scoring (closer to benchmark = higher score).	CTEs 5–10
PHASE 3	Weighted composite & ranking Configurable pillar weights, PERCENT_RANK() window functions for department-level ranking. Final output: candidate_id, composite_score, rank, tier classification.	CTEs 11–17

Key SQL Techniques Used

```
-- Benchmark extraction: median score per pillar WITH top_performers AS ( SELECT * FROM employees WHERE rating = 5 ), benchmark AS (
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY pillar_score) AS pillar_p50, MODE() WITHIN GROUP (ORDER BY education_field) AS
edu_mode FROM top_performers ) -- Final ranking with department-level percentile SELECT candidate_id, composite_score,
PERCENT_RANK() OVER (PARTITION BY department ORDER BY composite_score DESC) AS dept_rank FROM scored_candidates ORDER BY
composite_score DESC;
```

RESULT A ranked candidate matching system where every score component traces back to an explicit SQL rule — demonstrating analytical SQL depth beyond aggregation and joins.	DASHBOARD BEHAVIOR Manager selects a role vacancy → system computes match scores for all 2,010 candidates → ranked output with tier classification, plus an AI-generated job profile summary per vacancy.	DATA QUALITY Employees with missing ratings are excluded, not imputed. All 17 CTEs are documented with inline comments; a star schema separates candidate facts from assessment dimensions.
---	--	--

Stack: PostgreSQL · advanced SQL (CTE, window functions, PERCENTILE_CONT) · Streamlit · OpenRouter

Student Dropout *Early Warning System*

[➤ Live Demo — Streamlit App](#)[■ Looker Studio Dashboard](#)[↗ GitHub Repository](#)[≡ Case study](#)

92.42%
Accuracy
on test set

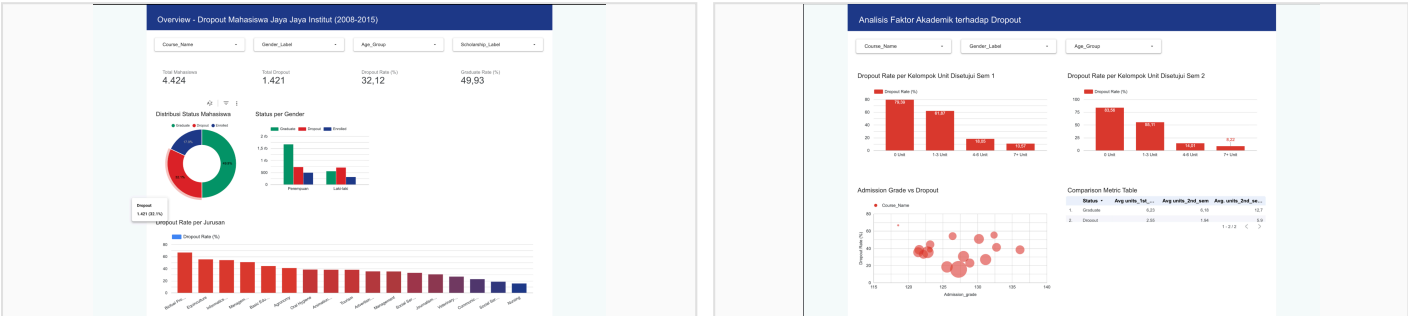
97.27%
ROC-AUC
XGBoost

32.12%
Baseline dropout rate
problem being solved

SMOTE
Class imbalance
handling

Context: An institution with a **32.12% dropout rate** was identifying at-risk students only after they had already left. **Problem:** Predict dropout early enough for intervention, and deliver the output in a form counselors would actually use.

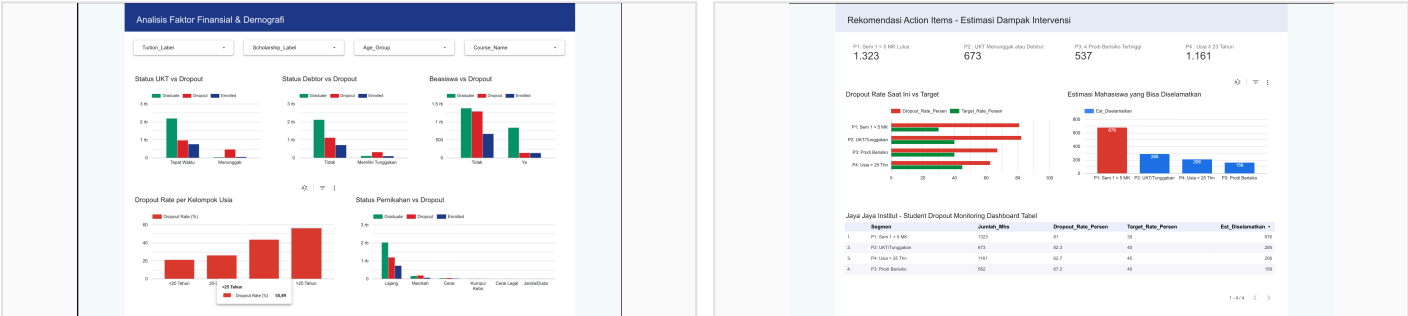
LOOKER STUDIO DASHBOARD — OVERVIEW & ACADEMIC PERFORMANCE



Tab 1 — Overview: dropout rate distribution by program, enrollment trends, and Graduate / Dropout / Enrolled breakdown across all faculties.

Tab 2 — Academic Performance: units passed in semester 1 & 2 vs. dropout rate. Students with 0–4 units passed in semester 1 show 74–88% dropout rate.

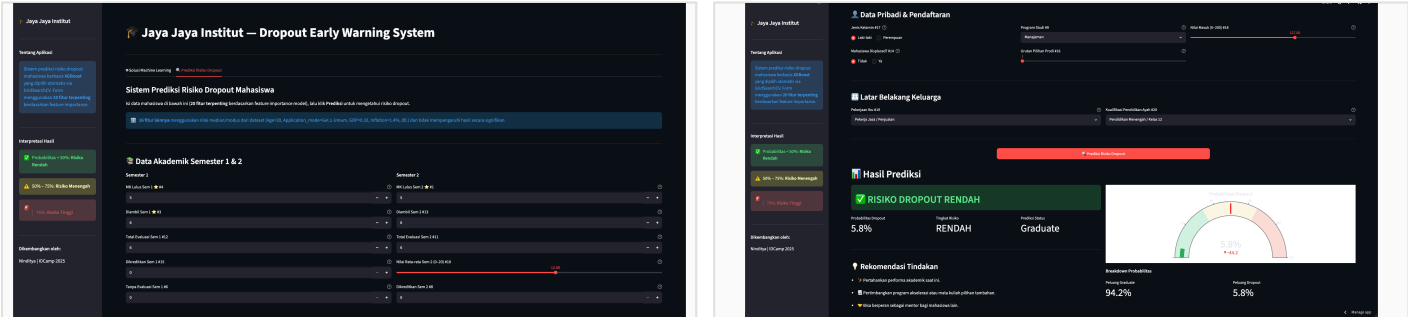
LOOKER STUDIO DASHBOARD — FINANCIAL FACTORS & ACTION ITEMS



Tab 3 — Financial & Demographic: tuition payment status, debtor impact, scholarship effect, dropout rate by age group and marital status — key operational signals for counselors.

Tab 4 — Action Items: priority intervention segments (P1–P4); ~1,325 of 1,421 at-risk students reachable, with before/after dropout-rate estimates per segment.

STREAMLIT PREDICTION APP — LIVE DEMO



Input form: 20-field form collecting academic, financial, and demographic signals for a single student.

Prediction result: dropout probability score with targeted intervention recommendations for the counselor.

RESULT

92.42% accuracy and **97.27% ROC-AUC**, with academic performance and tuition payment status identified as the dominant dropout drivers, converting a prediction into a specific intervention target.

TWO AUDIENCES, TWO SURFACES

Looker Studio for institutional management (trend monitoring, faculty-level overview) · **Streamlit app** for individual counselors (per-student risk score + actionable recommendations).

CLASS IMBALANCE

SMOTE applied to training data only, not the full dataset — prevents leakage while forcing the model to learn the minority dropout class. Recall prioritized in threshold tuning over raw accuracy.

Employee Attrition *Early Warning System*

[↶ GitHub Repository](#)

≡ Case study

86.11%
Recall on test set catches most leavers

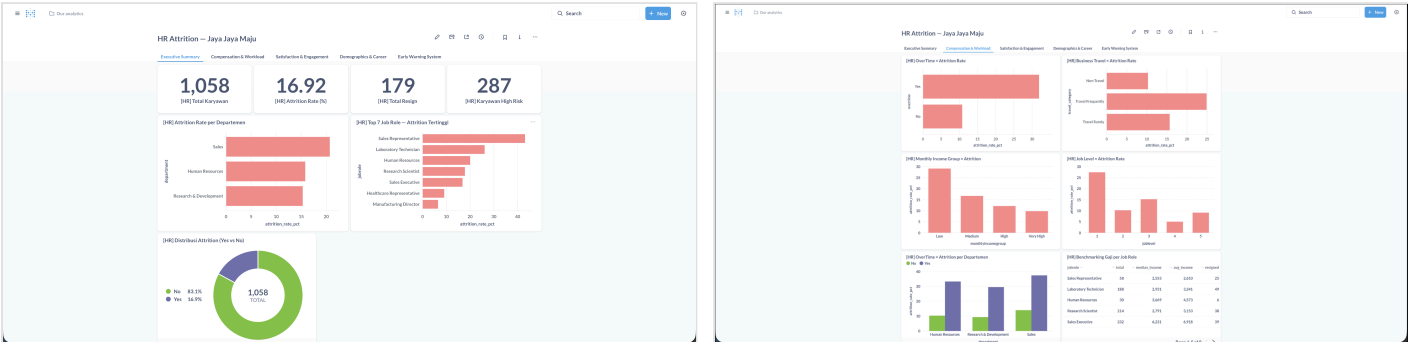
3x
Overtime risk multiplier
31.92% vs 10.79%

5
Metabase dashboard tabs
Docker deployed

16.92%
Annual attrition rate
baseline problem

Context: Attrition was being reported as a monthly percentage with no attribution to cause, leaving retention effort unfocused. **Problem:** Identify which operational conditions actually drive attrition, and quantify them well enough to justify a policy change.

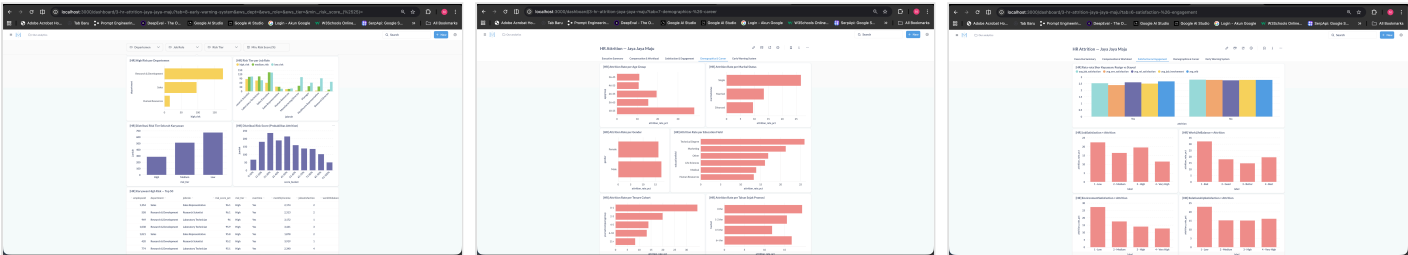
METABASE OPERATIONAL DASHBOARD – EXECUTIVE SUMMARY & COMPENSATION ANALYSIS



Tab 1 — Executive Summary: total employees, 16.92% attrition rate, department breakdown, top job roles by attrition rate — filterable by department and role.

Tab 2 — Compensation & Workload: overtime vs. attrition rate (3x multiplier confirmed), business travel impact, monthly income band analysis.

METABASE DASHBOARD – DEMOGRAPHICS, SATISFACTION & EARLY WARNING



Tab 3 — Early Warning: prediction results by department, high-risk employee flagging, prediction probability distribution for HR intervention prioritization.

Tab 4 — Demographics & Career: attrition by age group, gender, education field, tenure cohort, and years since last promotion.

Tab 5 — Satisfaction & Engagement: self-reported job satisfaction, work-life balance, environment and relationship scores vs. attrition rate.

RESULT

86.11% recall, with overtime identified as an approximately **3x risk multiplier** alongside compensation, business travel, and tenure. Output framed as proactive retention actions rather than a model score.

RECALL-FIRST DECISION

Recall chosen as the primary metric over raw accuracy: missing a leaver (false negative) costs an employee lost without intervention, while a false positive costs only a counseling conversation.

INFRASTRUCTURE

Docker Compose orchestrates PostgreSQL + Metabase containers — the full analytics environment spins up with a single command. Prediction results are written directly back to the database for dashboard consumption.

Stack: Python · XGBoost · Metabase · Docker · PostgreSQL

CineTrack Cinema Analytics Backend

PostgreSQL · Fastify · Redis

⌵ Backend Repository	⌵ Frontend Repository	≡ Case study	
5 Relational tables behind the analytics layer	60 Selected of 900+ program applicants	Redis Caching on highest- frequency endpoints	3 Analytics domains revenue · occupancy · demand

Context: Capstone project for a cinema chain requiring an analytics layer behind an operations dashboard. Role held: backend and SQL analytics.
Problem: Serve revenue, occupancy, and time-slot demand analytics across five relational tables at dashboard-acceptable latency.

ANALYTICS API ARCHITECTURE

POSTGRESQL CTES → FASTIFY API → REDIS CACHE			
QUERY	CTE-based analytical queries Revenue, occupancy, and time-slot demand computed via PostgreSQL CTE chains across the five-table relational schema (screenings, bookings, seats, theaters, films).	PostgreSQL	
SERVE	Fastify API layer Analytics endpoints exposed with Swagger documentation for frontend and dashboard consumption.	Node.js · Fastify	
OPTIMIZE	Redis caching & query optimization Highest-frequency endpoints (revenue and occupancy summaries) cached to keep response times at dashboard-acceptable latency under repeated polling.	Redis	

RESULT	RELEVANCE TO THIS ROLE	TEAM CONTEXT
Working analytics API supporting revenue, occupancy, and demand-trend reporting. Selected among 60 of 900+ applicants to the Korea-ASEAN Digital Academy capstone program.	Demonstrates backend-adjacent SQL analytics work — building the query layer a dashboard actually depends on, not just the dashboard itself.	Backend repository maintained under the team capstone organization; frontend repository maintained independently.

Stack: PostgreSQL · Node.js (Fastify) · Redis · Swagger

↙ GitHub Repository	≡ Main Notebook	≡ Full Write-up (README)
---------------------	-----------------	--------------------------

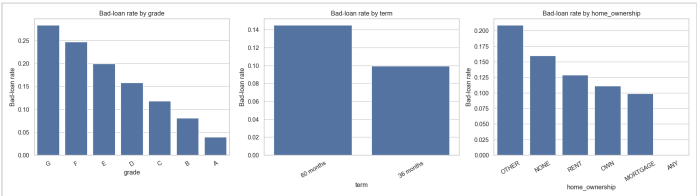
466K Loan records 75 raw columns audited	11.2% Baseline bad-loan rate the imbalance problem	0.705 Out-of-time ROC-AUC chronological validation	2.4× Risk multiplier grade G · 60-month segment
---	---	---	--

Context: Approving a high-risk loan risks the principal; rejecting a sound applicant loses interest income and a good customer — credit risk scoring has to get both directions right. **Problem:** Build a GOOD/BAD classifier using only information available at origination (not post-disbursement outcomes), on a target that is only ~11.2% BAD, and translate the output into underwriting decisions rather than a single accuracy number.

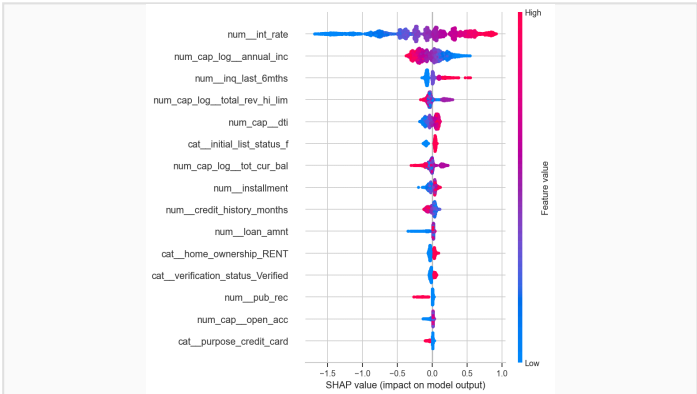
METHODOLOGY — LEAK-FREE BY CONSTRUCTION, THRESHOLD AS A BUSINESS DECISION

FEATURE AUDIT → THRESHOLD TUNING → OUT-OF-TIME VALIDATION			
AUDIT	75 raw columns disposition-audited, imbalance compared head-to-head Post-disbursement fields (total_pymnt, recoveries, out_prncp) dropped; outlier caps fit on the training fold only. class_weight/scale_pos_weight tested directly against SMOTE across 6 model variants.		75 → 27 features
THRESHOLD	F1-optimal vs. cost-based decision threshold F1-optimal threshold (0.585) tuned on out-of-fold probabilities; a separate expected-loss threshold (PD × LGD × EAD) tested at 0.070 — same review-capacity trade-off logic as the routing threshold in Case 2.		2 thresholds compared
VALIDATE	Chronological split + PSI drift check Train on 2007–Aug 2014 originations, test on Aug–Dec 2014 — ROC-AUC held at 0.705 (vs. 0.694 on a random split), confirming the model generalizes forward in time rather than memorizing an economic period.		PSI = 0.11

WHERE RISK CONCENTRATES & WHAT DRIVES THE SCORE — GRADE/TENOR BAD RATE & SHAP



Bad-loan rate rises monotonically from grade A (~4%) to grade G (~28%). 60-month loans and non-mortgage borrowers show consistently higher bad rates — the basis for the tenor-based pricing recommendation below.

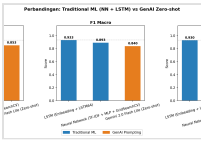


SHAP beeswarm (XGBoost): per-prediction, additive attribution — high `int_rate` sharply raises predicted risk, high income is protective. Same driver ranking a Logistic Regression coefficient view confirms independently.

BUSINESS RECOMMENDATION Grade G / 60-month loans carry a 26.6% bad rate — 2.4x the portfolio baseline — just 2.8% of volume but 6.5% of all bad loans in test. Recommendation: a tenor-based risk premium (+2–3pp) and mandatory manual underwriting review for this segment, prioritized because the low volume keeps review cost small relative to the loss it prevents.	SELF-CORRECTION, PROVEN NOT ASSUMED Retraining the same model with post-disbursement columns reattached pushes accuracy to 95.4% — confirming the reported 70–74% accuracy is genuine signal, not leakage. Calibration curves separately show raw probabilities run high (mean ≈0.46 vs. an actual 11.2% base rate) — ranking and threshold decisions stay valid, but raw probabilities need calibration before being read as literal PDs.	THRESHOLD IS A BUSINESS LEVER, NOT A CONSTANT The cost-minimizing threshold (0.070) flags ~89% of applications for review at that illustrative loss ratio — mathematically correct but operationally unworkable without an explicit review-capacity constraint. Documented as a next step rather than silently defaulted to 0.5.
---	---	--

Stack: Python · pandas · scikit-learn · XGBoost · imbalanced-learn (SMOTE) · SHAP · Flask

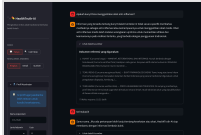
Work not selected as a primary case study but relevant to breadth of method — ML vs. GenAI benchmarking, production-pattern RAG services, statistical dashboarding, and QA automation.



Sentiment Analysis: ML vs. GenAI

Bidirectional LSTM and TF-IDF benchmarked against zero-shot GenAI on 11,000 rows of Indonesian Twitter text. **Conventional ML won on accuracy, latency, and annual cost simultaneously.** Published after self-correcting a KFold data leakage issue that had inflated the original result.

ninditya.github.io/sentiment-analysis



HealthTruth RAG Chatbot Service

Production-pattern RAG API with FastAPI, LangGraph, and Qdrant vector search, supporting four response modes over a grounded knowledge base. The second of the two public systems referenced in Case 1's evaluation methodology.

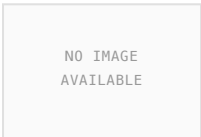
ninditya.github.io/rag-chatbot-service



Beijing Air Quality Dashboard

46MB of sensor data across 12 stations and 5 years reduced to a single decision dashboard with AQI computation, seasonal trend analysis, chi-square testing, and geospatial priority mapping.

ninditya.github.io/air-quality-dashboard



QA & Test Automation, SauceDemo

29 regression tests written in Playwright, Cypress, and Selenium against a live site, including a cross-framework bug investigation where two frameworks disagreed on the same defect.

ninditya.github.io/saucedemo-qa

Full portfolio with interactive case studies: ninditya.github.io